Lekcja 08: Funkcje i idiomy

`#lekcja` `#datascience` `#python` `#funkcje` `#lambda` `#comprehensions`

W tej lekcji poznamy kluczowe techniki pisania modularnego i efektywnego kodu w Pythonie. Nauczymy się definiować własne funkcje, wykorzystywać funkcje lambda, stosować comprehensions do przetwarzania kolekcji oraz poznamy idiomatyczne wzorce Pythona używane w Data Science. Te umiejętności pozwolą nam pisać kod wielokrotnego użytku, który jest łatwiejszy w utrzymaniu i bardziej efektywny obliczeniowo.

---

1. Funkcje - podstawy modularnego programowania

Definition

Funkcja to nazwany blok kodu, który wykonuje określone zadanie i może być wywoływany wielokrotnie z różnymi danymi wejściowymi (argumentami). Funkcje są podstawą modularnego programowania i pozwalają na redukcję duplikacji kodu.

Wprowadzenie

Funkcje są fundamentalnym narzędziem w Data Science. Pozwalają nam enkapsulować logikę obliczeń, która może być używana wielokrotnie w różnych miejscach projektu. Zamiast powielać ten sam kod, definiujemy funkcję raz i wywołujemy ją w razie potrzeby. To podejście:

- **Zmniejsza duplikację kodu** - ta sama logika w jednym miejscu
- **Ułatwia utrzymanie** - zmiany wymagają edycji tylko w jednym miejscu
- **Poprawia czytelność** - nazwane funkcje dokumentują swoje przeznaczenie
- **Redukuje błędy** - przetestowany kod można używać wielokrotnie
- **Oszczędza zasoby** - mniej kodu do przetworzenia przez interpreter

Przykład 1: Definiowanie prostych funkcji - obliczenia statystyczne

```
# Definiowanie funkcji bez parametrów
def powitanie():
    """Wyświetla wiadomość powitalną."""
    print("Witaj w analizie danych!")

# Wywołanie funkcji
```

```python
powitanie()  # Wynik: Witaj w analizie danych!

# Funkcja z parametrami - obliczanie średniej
def oblicz_srednia(lista_liczb):
    """
    Oblicza średnią arytmetyczną z listy liczb.

    Args:
        lista_liczb (list): Lista wartości numerycznych

    Returns:
        float: Średnia arytmetyczna
    """
    suma = 0
    for liczba in lista_liczb:
        suma += liczba
    srednia = suma / len(lista_liczb)
    return srednia

# Dane testowe - wyniki sprzedaży w tys. PLN
sprzedaz = [120.5, 145.2, 98.7, 167.3, 134.6]

# Wywołanie funkcji i zapisanie wyniku
srednia_sprzedaz = oblicz_srednia(sprzedaz)
print(f"Średnia sprzedaż: {srednia_sprzedaz:.2f} tys. PLN")  # 133.26 tys.
PLN
```

Tip

Używaj docstringów (potrójnych cudzysłowów) do dokumentowania funkcji. To pomaga innym (i Tobie w przyszłości) zrozumieć, co robi funkcja, jakie przyjmuje argumenty i co zwraca.

Przykład 2: Funkcje z wieloma parametrami - preprocessing danych

```python
# Funkcja z wieloma parametrami do czyszczenia danych
def przygotuj_dane(lista_wartosci, min_wartosc, max_wartosc):
    """
    Filtruje listę wartości, pozostawiając tylko te w zadanym zakresie.

    Args:
        lista_wartosci (list): Lista liczb do przefiltrowania
        min_wartosc (float): Minimalna akceptowalna wartość
        max_wartosc (float): Maksymalna akceptowalna wartość
```

```
    Returns:
        list: Przefiltrowana lista wartości
    """
    czyste_dane = []
    for wartosc in lista_wartosci:
        if min_wartosc <= wartosc <= max_wartosc:
            czyste_dane.append(wartosc)
    return czyste_dane

# Dane z czujników temperatury (niektóre błędne)
temperatury = [22.1, 23.5, -999, 24.2, 25.1, 100, 23.8, 22.9]

# Filtrowanie danych - akceptujemy tylko temperatury 15-35°C
temperatury_ok = przygotuj_dane(temperatury, 15, 35)
print(f"Oryginalne dane: {len(temperatury)} pomiarów")
print(f"Po czyszczeniu: {len(temperatury_ok)} pomiarów")
print(f"Czyste dane: {temperatury_ok}")
# Wynik: [22.1, 23.5, 24.2, 25.1, 23.8, 22.9]
```

Przykład 3: Funkcje z wartościami domyślnymi - analiza danych

```
# Funkcja z parametrami domyślnymi
def normalizuj_dane(dane, metoda="min-max", zaokraglenie=2):
    """
    Normalizuje dane przy użyciu wybranej metody.

    Args:
        dane (list): Lista wartości do normalizacji
        metoda (str): Metoda normalizacji ("min-max" lub "z-score")
        zaokraglenie (int): Liczba miejsc po przecinku

    Returns:
        list: Znormalizowane dane
    """
    if metoda == "min-max":
        # Normalizacja min-max: (x - min) / (max - min)
        min_val = min(dane)
        max_val = max(dane)
        zakres = max_val - min_val

        znormalizowane = []
        for wartosc in dane:
            norma = (wartosc - min_val) / zakres
            znormalizowane.append(round(norma, zaokraglenie))
```

```
        return znormalizowane

    elif metoda == "z-score":
        # Normalizacja z-score: (x - średnia) / odchylenie standardowe
        # Obliczamy średnią
        srednia = sum(dane) / len(dane)

        # Obliczamy odchylenie standardowe
        suma_kwadratow = sum((x - srednia) ** 2 for x in dane)
        odchylenie = (suma_kwadratow / len(dane)) ** 0.5

        znormalizowane = []
        for wartosc in dane:
            z = (wartosc - srednia) / odchylenie
            znormalizowane.append(round(z, zaokraglenie))
        return znormalizowane

# Dane - ceny produktów w PLN
ceny = [49.99, 89.99, 129.99, 199.99, 299.99]

# Normalizacja min-max (domyślna)
ceny_norm_minmax = normalizuj_dane(ceny)
print(f"Oryginalne ceny: {ceny}")
print(f"Normalizacja min-max: {ceny_norm_minmax}")
# Wynik: [0.0, 0.16, 0.32, 0.6, 1.0]

# Normalizacja z-score
ceny_norm_zscore = normalizuj_dane(ceny, metoda="z-score", zaokraglenie=3)
print(f"Normalizacja z-score: {ceny_norm_zscore}")
# Wynik: [-1.289, -0.541, 0.207, 0.956, 1.667]
```

Note

Parametry domyślne pozwalają na elastyczne wywoływanie funkcji. Możemy pominąć argumenty, które mają sensowne wartości domyślne, lub je nadpisać w razie potrzeby.

Przykład 4: Funkcje zwracające wiele wartości - statystyki opisowe

```
# Funkcja zwracająca tuple z wieloma wynikami
def statystyki_opisowe(dane):
    """
    Oblicza podstawowe statystyki opisowe dla danych.

    Args:
```

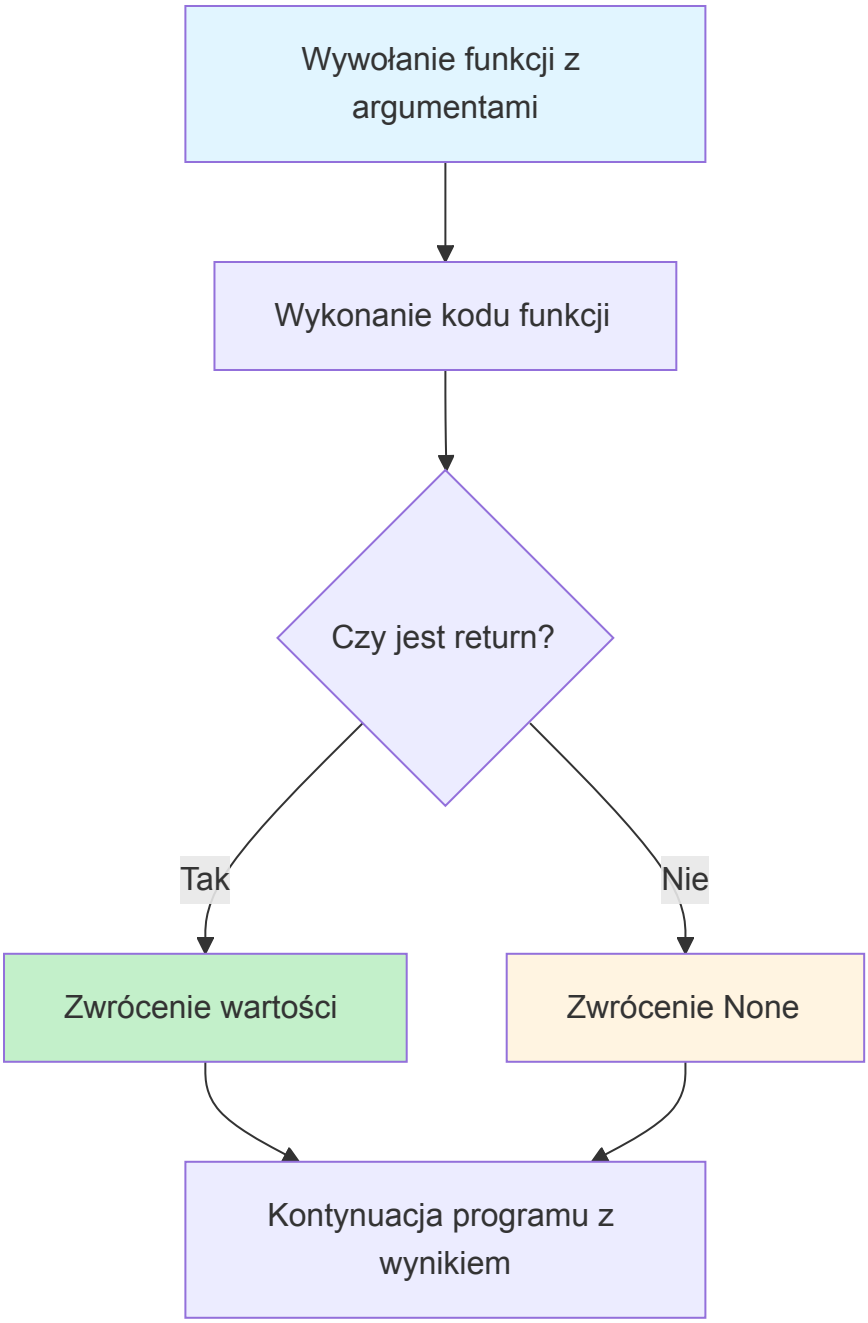
```
        dane (list): Lista wartości numerycznych

    Returns:
        tuple: (średnia, mediana, minimum, maksimum, zakres)
    """
    # Średnia
    srednia = sum(dane) / len(dane)

    # Mediana (sortujemy i bierzemy środkowy element)
    posortowane = sorted(dane)
    n = len(posortowane)
    if n % 2 == 0:
        mediana = (posortowane[n//2 - 1] + posortowane[n//2]) / 2
    else:
        mediana = posortowane[n//2]

    # Minimum i maksimum
    minimum = min(dane)
    maksimum = max(dane)

    # Zakres (rozstęp)
    zakres = maksimum - minimum

    # Zwracamy wszystkie wyniki jako tuple
    return srednia, mediana, minimum, maksimum, zakres

# Dane - dzienny czas pracy w godzinach
czas_pracy = [8.5, 7.2, 9.1, 8.0, 6.5, 10.2, 7.8, 8.3, 9.5, 7.9]

# Rozpakowywanie zwróconych wartości
sr, med, mini, maxi, zak = statystyki_opisowe(czas_pracy)

print("Statystyki czasu pracy:")
print(f"  Średnia: {sr:.1f}h")
print(f"  Mediana: {med:.1f}h")
print(f"  Minimum: {mini:.1f}h")
print(f"  Maksimum: {maxi:.1f}h")
print(f"  Zakres: {zak:.1f}h")
```

Wizualizacja: Porównanie funkcji przed i po normalizacji

```
# Symulacja danych do wizualizacji
import random

# Generujemy próbkę danych - wynagrodzenia w tys. PLN
```

```
random.seed(42)
wynagrodzenia = [round(random.uniform(3, 15), 1) for _ in range(50)]

# Normalizujemy dane
wynagrodzenia_norm = normalizuj_dane(wynagrodzenia, metoda="min-max",
zaokraglenie=3)

# Wizualizacja (prosty wykres ASCII)
print("\n=== Porównanie danych przed i po normalizacji ===\n")
print("Oryginalne dane (próbka 10 wartości):")
print(wynagrodzenia[:10])
print(f"Zakres: {min(wynagrodzenia):.1f} - {max(wynagrodzenia):.1f} tys.
PLN")

print("\nZnormalizowane dane (próbka 10 wartości):")
print(wynagrodzenia_norm[:10])
print(f"Zakres: {min(wynagrodzenia_norm)} - {max(wynagrodzenia_norm)}")
```

Schemat działania funkcji



⚠ Warning

Jeśli funkcja nie zawiera instrukcji `return`, zwraca ona wartość `None`. To może prowadzić do błędów, jeśli próbujesz użyć wyniku takiej funkcji w dalszych obliczeniach.

---

2. Funkcje lambda - jednolinijkowe funkcje anonimowe

Definition

Funkcja lambda to anonimowa funkcja jednolinijkowa, definiowana przy użyciu słowa kluczowego `lambda`. Jest używana do prostych operacji, które nie wymagają pełnej definicji funkcji.

Wprowadzenie

Funkcje lambda są szczególnie przydatne w Data Science przy:

- Transformacjach danych w jednej linii
- Przekazywaniu prostych funkcji jako argumentów (np. do `sorted()`, `map()`, `filter()`)
- Feature engineeringu - tworzeniu nowych cech z istniejących
- Operacjach na DataFrame'ach w Pandas (przyszła lekcja)

Składnia: `lambda argumenty: wyrażenie`

Przykład 1: Podstawowe funkcje lambda - transformacje danych

```
# Tradycyjna funkcja
def podwoj(x):
    return x * 2

# Równoważna funkcja lambda
podwoj_lambda = lambda x: x * 2

# Porównanie użycia
liczba = 5
print(f"Tradycyjna funkcja: {podwoj(liczba)}")      # 10
print(f"Funkcja lambda: {podwoj_lambda(liczba)}")   # 10

# Lambda z wieloma argumentami - obliczanie BMI
bmi = lambda waga, wzrost: waga / (wzrost ** 2)

# Dane pacjentów
waga_kg = 75
wzrost_m = 1.80

wskaznik_bmi = bmi(waga_kg, wzrost_m)
print(f"\nBMI: {wskaznik_bmi:.1f}")  # 23.1

# Lambda do konwersji jednostek - Celsius na Fahrenheit
c_to_f = lambda celsius: celsius * 9/5 + 32

temperatury_c = [0, 10, 20, 30, 37]
```

```
print("\nKonwersja temperatur C → F:")
for temp_c in temperatury_c:
    temp_f = c_to_f(temp_c)
    print(f"  {temp_c}°C = {temp_f:.1f}°F")
```

Przykład 2: Lambda z sorted() - sortowanie złożonych struktur

```
# Dane - lista studentów (imię, wiek, średnia)
studenci = [
    ("Anna", 22, 4.5),
    ("Bartosz", 21, 4.2),
    ("Celina", 23, 4.8),
    ("Damian", 22, 4.1),
    ("Eliza", 21, 4.6)
]

# Sortowanie po imieniu (domyślnie)
posortowani_imie = sorted(studenci)
print("Sortowanie po imieniu:")
for student in posortowani_imie:
    print(f"  {student}")

# Sortowanie po wieku (drugi element tuple)
posortowani_wiek = sorted(studenci, key=lambda s: s[1])
print("\nSortowanie po wieku:")
for student in posortowani_wiek:
    print(f"  {student[0]}, wiek: {student[1]}")

# Sortowanie po średniej (trzeci element) - malejąco
posortowani_srednia = sorted(studenci, key=lambda s: s[2], reverse=True)
print("\nRanking według średniej:")
for i, student in enumerate(posortowani_srednia, 1):
    print(f"  {i}. {student[0]} - średnia: {student[2]}")
```

Przykład 3: Lambda z map() i filter() - przetwarzanie kolekcji

```
# map() - aplikuje funkcję do każdego elementu
# Dane - ceny netto produktów
ceny_netto = [100, 250, 450, 890, 1200]

# Dodajemy VAT 23% używając lambda i map()
ceny_brutto = list(map(lambda cena: cena * 1.23, ceny_netto))

print("Konwersja cen netto → brutto (VAT 23%):")
```

```python
for netto, brutto in zip(ceny_netto, ceny_brutto):
    print(f"  {netto} PLN → {brutto:.2f} PLN")

# filter() - filtruje elementy spełniające warunek
# Dane - wyniki testów
wyniki = [45, 67, 89, 34, 78, 92, 56, 81, 43, 88]

# Filtrujemy tylko wyniki ≥ 60 (zdane testy)
zdane = list(filter(lambda wynik: wynik >= 60, wyniki))

print(f"\nWszystkie wyniki: {wyniki}")
print(f"Wyniki zdane (≥60): {zdane}")
print(f"Procent zdawalności: {len(zdane) / len(wyniki) * 100:.1f}%")

# Łączenie map() i filter() - przetwarzanie wieloetapowe
# Dane - czas reakcji w milisekundach (niektóre błędne)
czasy_ms = [120, 145, 2000, 134, 98, 156, -50, 178, 3500, 142]

# Etap 1: filtrujemy tylko prawidłowe czasy (50-500 ms)
czasy_ok = filter(lambda t: 50 <= t <= 500, czasy_ms)

# Etap 2: konwertujemy na sekundy
czasy_s = list(map(lambda t: t / 1000, czasy_ok))

print(f"\nCzasy reakcji (s): {czasy_s}")
print(f"Średni czas reakcji: {sum(czasy_s) / len(czasy_s):.3f}s")
```

Tip

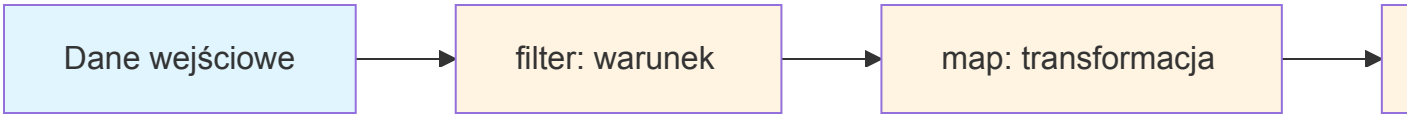
Funkcje `map()` i `filter()` zwracają iteratory (obiekty leniwe), które generują wartości na żądanie. Musisz je przekonwertować na listę używając `list()`, jeśli chcesz zobaczyć wszystkie wyniki.

Przykład 4: Lambda w kontekście Data Science - feature engineering

```python
# Dane klientów - wiek, dochód, wydatki miesięczne
klienci = [
    {"id": 1, "wiek": 25, "dochod": 4500, "wydatki": 3200},
    {"id": 2, "wiek": 35, "dochod": 7800, "wydatki": 5100},
    {"id": 3, "wiek": 45, "dochod": 9200, "wydatki": 4800},
    {"id": 4, "wiek": 28, "dochod": 5200, "wydatki": 4900},
    {"id": 5, "wiek": 52, "dochod": 11000, "wydatki": 6200}
]
```

```python
# Feature engineering - tworzymy nową cechę: wskaźnik oszczędności
# Oszczędności = (dochód - wydatki) / dochód
dodaj_wskaznik_oszczednosci = lambda k: {
    **k,  # rozpakowujemy istniejące pola
    "wskaznik_oszczednosci": round((k["dochod"] - k["wydatki"]) /
k["dochod"], 2)
}

klienci_z_wskaznikiem = list(map(dodaj_wskaznik_oszczednosci, klienci))

print("Klienci z wskaźnikiem oszczędności:")
for klient in klienci_z_wskaznikiem:
    print(f"  ID {klient['id']}: dochód={klient['dochod']}, "
          f"wydatki={klient['wydatki']}, "
          f"wskaźnik={klient['wskaznik_oszczednosci']}")

# Segmentacja klientów - wysoki wskaźnik oszczędności (≥0.30)
oszczedni_klienci = list(filter(
    lambda k: k["wskaznik_oszczednosci"] >= 0.30,
    klienci_z_wskaznikiem
))

print(f"\nKlienci oszczędni (wskaźnik ≥30%): {len(oszczedni_klienci)}")
```

Wizualizacja: Pipeline przetwarzania z lambda



Warning

Funkcje lambda mogą zawierać tylko jedno wyrażenie. Nie możesz używać instrukcji wieloliniowych, pętli ani złożonej logiki. Dla bardziej skomplikowanych operacji użyj zwykłej funkcji.

---

3. List Comprehensions - idiomatyczne tworzenie list

Definition

List comprehension to zwięzła składnia Pythona do tworzenia nowych list na podstawie istniejących sekwencji. Pozwala na transformację i filtrowanie danych w jednej linii kodu.

Wprowadzenie

List comprehensions to jeden z najbardziej charakterystycznych idiomów Pythona. W Data Science są niezwykle przydatne do:

- Szybkiego przetwarzania danych
- Tworzenia cech (feature engineering)
- Czyszczenia i transformacji danych
- Filtrowania obserwacji

Składnia: `[wyrażenie for element in sekwencja if warunek]`

Przykład 1: Podstawowe comprehensions - transformacje

```
# Tradycyjna pętla for
kwadraty_petla = []
for i in range(1, 11):
    kwadraty_petla.append(i ** 2)

# List comprehension - ten sam rezultat, krótsza składnia
kwadraty_comp = [i ** 2 for i in range(1, 11)]

print("Tradycyjna pętla:")
print(kwadraty_petla)
print("\nList comprehension:")
print(kwadraty_comp)

# Dane rzeczywiste - konwersja temperatur
temperatury_fahrenheit = [32, 50, 68, 86, 104]

# Konwersja F → C używając comprehension
temperatury_celsius = [(f - 32) * 5/9 for f in temperatury_fahrenheit]

print("\nKonwersja temperatur:")
for f, c in zip(temperatury_fahrenheit, temperatury_celsius):
    print(f"  {f}°F = {c:.1f}°C")

# Przetwarzanie stringów - ekstrakcja długości
miasta = ["Warszawa", "Kraków", "Wrocław", "Poznań", "Gdańsk"]
dlugosci_nazw = [len(miasto) for miasto in miasta]
```

```
print(f"\nMiasta: {miasta}")
print(f"Długości nazw: {dlugosci_nazw}")
```

Przykład 2: Comprehensions z warunkami - filtrowanie danych

```
# Filtrowanie liczb parzystych
liczby = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
parzyste = [x for x in liczby if x % 2 == 0]

print(f"Wszystkie liczby: {liczby}")
print(f"Tylko parzyste: {parzyste}")

# Dane - wyniki sprzedaży w tys. PLN
sprzedaz_dzienna = [12.5, 8.3, 15.7, 9.1, 22.4, 6.8, 18.9, 14.2, 11.6, 7.5]

# Filtrujemy tylko dni z wysoką sprzedażą (>10 tys.)
wysoka_sprzedaz = [s for s in sprzedaz_dzienna if s > 10]

print(f"\nWszystkie dni: {len(sprzedaz_dzienna)}")
print(f"Dni z wysoką sprzedażą (>10k): {len(wysoka_sprzedaz)}")
print(f"Wartości: {wysoka_sprzedaz}")
print(f"Średnia w dniach wysokiej sprzedaży: {sum(wysoka_sprzedaz) /
len(wysoka_sprzedaz):.2f} tys.")

# Filtrowanie i transformacja jednocześnie
# Dane - oceny studentów
oceny = [3.0, 4.5, 2.5, 5.0, 3.5, 4.0, 2.0, 4.5, 5.0, 3.0]

# Wybieramy tylko oceny ≥3.5 i zamieniamy na procenty (ocena * 20)
dobre_oceny_procent = [ocena * 20 for ocena in oceny if ocena >= 3.5]

print(f"\nOceny ≥3.5 w procentach: {dobre_oceny_procent}")
```

Przykład 3: Zagnieżdżone comprehensions - macierze i struktury 2D

```
# Tworzenie macierzy (lista list) - tabela mnożenia
rozmiar = 5
tabela_mnozenia = [[i * j for j in range(1, rozmiar + 1)] for i in range(1,
rozmiar + 1)]

print("Tabela mnożenia 5x5:")
for wiersz in tabela_mnozenia:
    # Formatowanie: każda liczba zajmuje 4 znaki
```

```
    print("  " + "  ".join(f"{x:3d}" for x in wiersz))

# Spłaszczanie (flattening) zagnieżdżonej listy
macierz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
splaaszczona = [element for wiersz in macierz for element in wiersz]

print(f"\nMacierz 3x3: {macierz}")
print(f"Spłaszczona: {splaaszczona}")

# Dane rzeczywiste - macierz odległości między miastami (km)
miasta = ["Warszawa", "Kraków", "Wrocław"]
odleglosci = [
    [0, 290, 350],      # z Warszawy
    [290, 0, 265],      # z Krakowa
    [350, 265, 0]       # z Wrocławia
]

# Wyciągamy wszystkie niezerowe odległości
wszystkie_odleglosci = [
    odl
    for wiersz in odleglosci
    for odl in wiersz
    if odl > 0
]

print(f"\nŚrednia odległość między miastami: {sum(wszystkie_odleglosci) /
len(wszystkie_odleglosci):.0f} km")
```

Przykład 4: Comprehensions w feature engineering - tworzenie cech

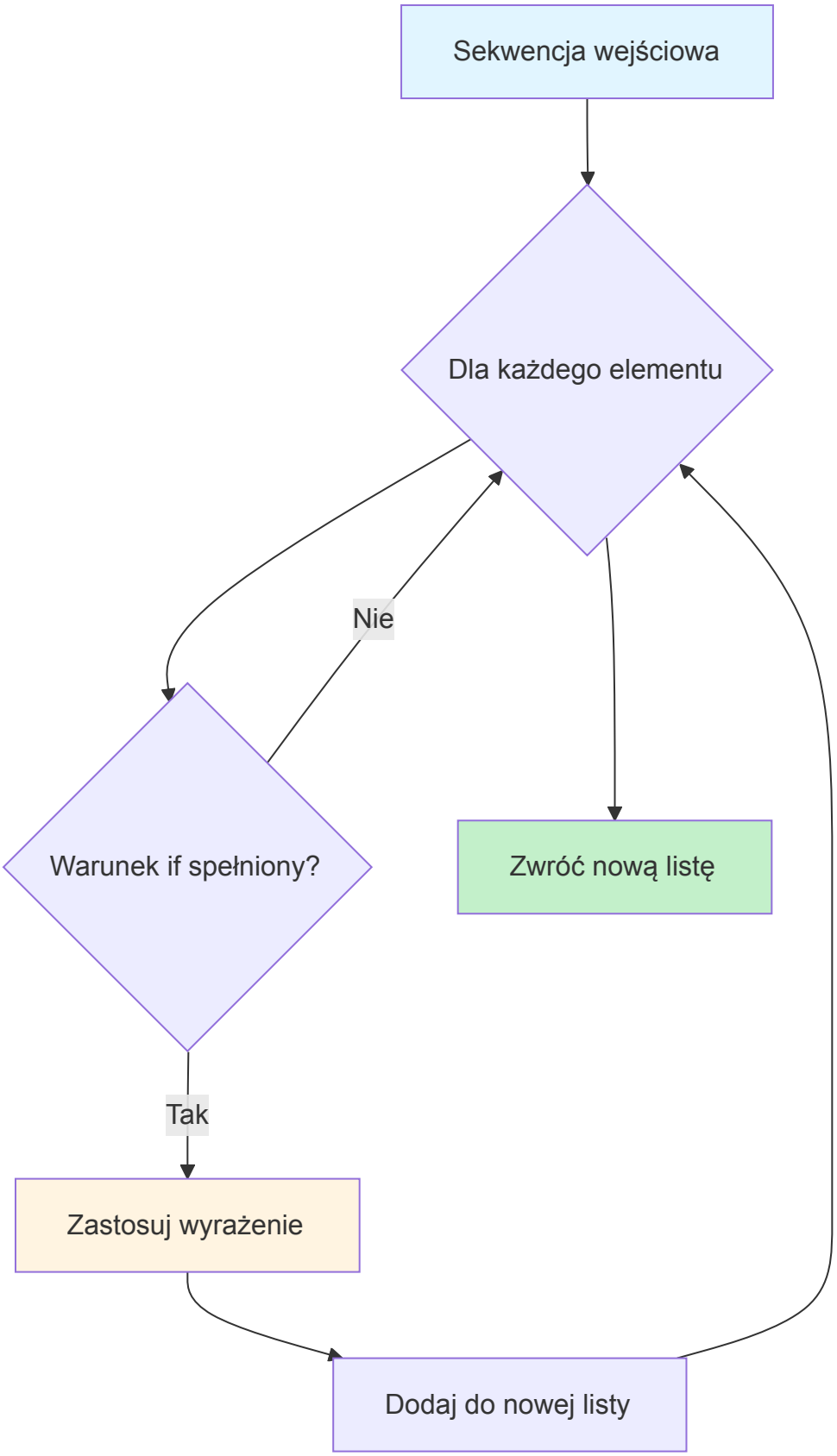
```
# Dane klientów - wiek i miesięczny dochód
klienci_raw = [
    (1, 25, 4500),   # (id, wiek, dochód)
    (2, 35, 7800),
    (3, 45, 9200),
    (4, 28, 5200),
    (5, 52, 11000),
    (6, 31, 6800),
    (7, 42, 8500),
    (8, 29, 5900)
]

# Feature engineering - kategorie wiekowe
def kategoria_wiekowa(wiek):
    if wiek < 30:
```

```
        return "mlody"
    elif wiek < 45:
        return "sredni"
    else:
        return "starszy"

# Feature engineering - poziom dochodów
def poziom_dochodu(dochod):
    if dochod < 5000:
        return "niski"
    elif dochod < 8000:
        return "sredni"
    else:
        return "wysoki"

# Tworzymy nowe cechy używając comprehension
klienci_engineered = [
    {
        "id": k[0],
        "wiek": k[1],
        "dochod": k[2],
        "kat_wiekowa": kategoria_wiekowa(k[1]),
        "poz_dochodu": poziom_dochodu(k[2]),
        "dochod_na_rok_zycia": round(k[2] / k[1], 2)
    }
    for k in klienci_raw
]

print("Klienci z dodatkowymi cechami:")
for klient in klienci_engineered[:3]:  # pokazujemy pierwszych 3
    print(f"  ID {klient['id']}: wiek={klient['wiek']}, "
          f"kat_wiekowa={klient['kat_wiekowa']}, "
          f"dochód={klient['dochod']}, "
          f"poz_dochodu={klient['poz_dochodu']}")

# Segmentacja - klienci młodzi z wysokim dochodem
segment_mlodzi_wysoki = [
    k for k in klienci_engineered
    if k["kat_wiekowa"] == "mlody" and k["poz_dochodu"] == "wysoki"
]

print(f"\nSegment 'młodzi + wysoki dochód': {len(segment_mlodzi_wysoki)}
klientów")
```

Wizualizacja: Porównanie wydajności pętla vs comprehension

```python
# Pomiar czasu wykonania (prosta symulacja)
import time

n = 100000  # liczba elementów

# Metoda 1: tradycyjna pętla
start = time.time()
wynik_petla = []
for i in range(n):
    wynik_petla.append(i * 2)
czas_petla = time.time() - start

# Metoda 2: list comprehension
start = time.time()
wynik_comp = [i * 2 for i in range(n)]
czas_comp = time.time() - start

print("\n=== Porównanie wydajności (n=100 000) ===")
print(f"Pętla for:         {czas_petla:.4f}s")
print(f"Comprehension:     {czas_comp:.4f}s")
print(f"Comprehension jest szybsze o: {(czas_petla - czas_comp) / czas_petla
* 100:.1f}%")
```

Schemat działania comprehension


Sekwencja wejściowa
Dla każdego elementu
Nie
Warunek if spełniony?
Zwróć nową listę
Tak
Zastosuj wyrażenie
Dodaj do nowej listy

✓ **Success**

List comprehensions są nie tylko krótsze i bardziej czytelne, ale też szybsze niż tradycyjne pętle for. Python optymalizuje je wewnętrznie, co przekłada się na lepszą wydajność.

---

4. Dictionary i Set Comprehensions - rozszerzenia idiomatyki

Definition

Dictionary comprehension i **set comprehension** działają analogicznie do list comprehension, ale tworzą odpowiednio słowniki i zbiory zamiast list.

Wprowadzenie

Comprehensions w Pythonie nie ograniczają się tylko do list. Możemy tworzyć również:

- **Dictionary comprehensions**: `{klucz: wartość for ... in ... if ...}`
- **Set comprehensions**: `{wyrażenie for ... in ... if ...}`

Te konstrukcje są równie wydajne i czytelne jak list comprehensions.

Przykład 1: Dictionary comprehensions - mapowanie danych

```
# Tworzenie słownika z list
produkty = ["laptop", "mysz", "klawiatura", "monitor"]
ceny = [2999, 89, 249, 899]

# Dictionary comprehension - mapowanie produkt → cena
cennik = {produkt: cena for produkt, cena in zip(produkty, ceny)}

print("Cennik:")
for produkt, cena in cennik.items():
    print(f"  {produkt}: {cena} PLN")

# Inwersja słownika - zamiana klucz ↔ wartość
cena_produkt = {cena: produkt for produkt, cena in cennik.items()}
print(f"\nInwersja - cena jako klucz: {cena_produkt}")

# Filtrowanie słownika - tylko produkty drogie (>500 PLN)
drogie_produkty = {prod: cena for prod, cena in cennik.items() if cena >
```

```
500}
print(f"\nDrogie produkty (>500 PLN): {drogie_produkty}")
```

Przykład 2: Dictionary comprehensions - agregacje i statystyki

```
# Dane - wyniki ankiet (1-5) od klientów dla różnych aspektów
odpowiedzi = [
    {"aspekt": "jakosc", "ocena": 4},
    {"aspekt": "cena", "ocena": 3},
    {"aspekt": "jakosc", "ocena": 5},
    {"aspekt": "obsluga", "ocena": 4},
    {"aspekt": "cena", "ocena": 4},
    {"aspekt": "jakosc", "ocena": 4},
    {"aspekt": "obsluga", "ocena": 5},
    {"aspekt": "cena", "ocena": 2}
]

# Grupujemy oceny po aspektach
from collections import defaultdict
grouped = defaultdict(list)
for odp in odpowiedzi:
    grouped[odp["aspekt"]].append(odp["ocena"])

# Dictionary comprehension - obliczamy średnią dla każdego aspektu
srednie_oceny = {
    aspekt: round(sum(oceny) / len(oceny), 2)
    for aspekt, oceny in grouped.items()
}

print("Średnie oceny aspektów:")
for aspekt, srednia in srednie_oceny.items():
    print(f"  {aspekt}: {srednia}")
```

Przykład 3: Set comprehensions - usuwanie duplikatów i operacje zbiorowe

```
# Dane - lista zakupów (z duplikatami)
zakupy = [
    "chleb", "mleko", "masło", "chleb", "jajka",
    "mleko", "ser", "chleb", "woda", "mleko"
]

# Set comprehension - unikalne produkty (duże litery)
unikalne_produkty = {produkt.upper() for produkt in zakupy}
```

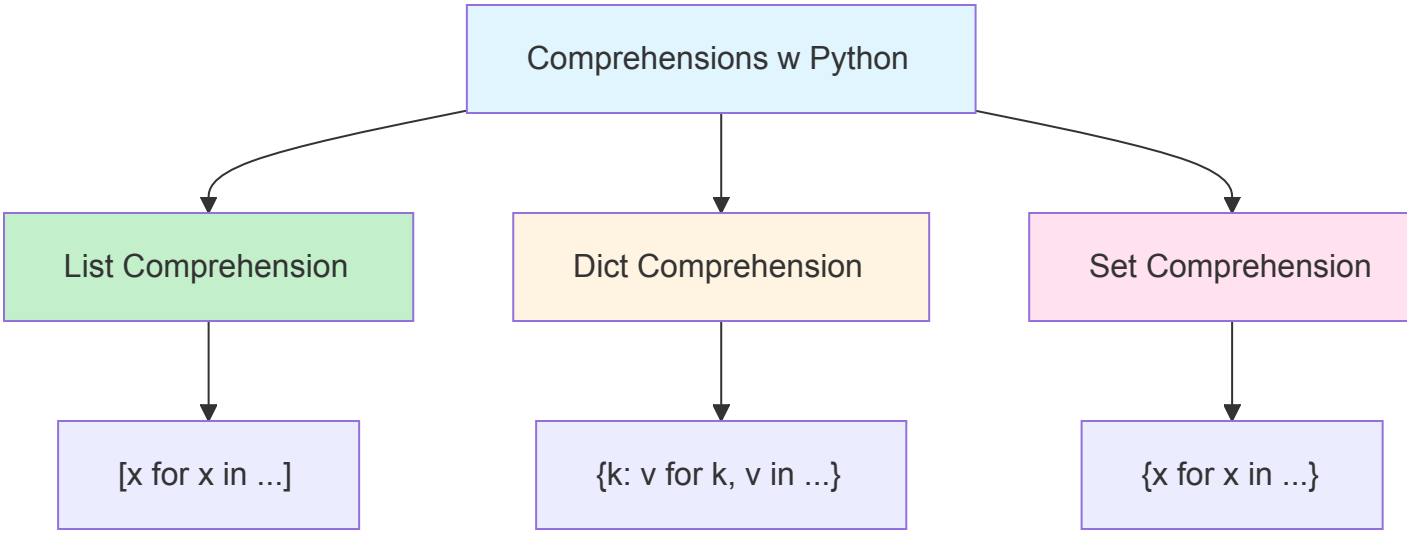
```python
print(f"Zakupy (z duplikatami): {zakupy}")
print(f"Unikalne produkty: {unikalne_produkty}")

# Dane - kategorie wiekowe uczestników
uczestnicy = [25, 32, 18, 45, 23, 35, 19, 52, 28, 41, 22, 38, 26, 48, 31]

# Set comprehension - unikalne kategorie (20-29, 30-39, 40-49, 50+)
kategorie = {(wiek // 10) * 10 for wiek in uczestnicy}
print(f"\nWiek uczestników: {uczestnicy}")
print(f"Reprezentowane kategorie wiekowe: {sorted(kategorie)}")

# Dane - tagi produktów
produkt1_tagi = {"elektronika", "komputer", "biuro", "praca"}
produkt2_tagi = {"elektronika", "audio", "rozrywka", "dom"}

# Set comprehension - wspólne tagi
wspolne = {tag for tag in produkt1_tagi if tag in produkt2_tagi}
print(f"\nWspólne tagi: {wspolne}")
```

Przykład 4: Kombinowanie comprehensions - złożone transformacje

```python
# Dane - transakcje sprzedażowe
transakcje = [
    {"id": 1, "produkt": "laptop", "cena": 2999, "ilosc": 2},
    {"id": 2, "produkt": "mysz", "cena": 89, "ilosc": 5},
    {"id": 3, "produkt": "klawiatura", "cena": 249, "ilosc": 3},
    {"id": 4, "produkt": "monitor", "cena": 899, "ilosc": 1},
    {"id": 5, "produkt": "laptop", "cena": 2999, "ilosc": 1}
]

# Dictionary comprehension - suma wartości dla każdego produktu
wartosci_produktow = {}
for trans in transakcje:
    prod = trans["produkt"]
    wartosc = trans["cena"] * trans["ilosc"]
    if prod in wartosci_produktow:
        wartosci_produktow[prod] += wartosc
    else:
        wartosci_produktow[prod] = wartosc

print("Całkowita wartość sprzedaży po produktach:")
for produkt, wartosc in wartosci_produktow.items():
    print(f"  {produkt}: {wartosc} PLN")

# Set comprehension - unikalne produkty drogie (cena jednostkowa >500)
```

```
drogie_produkty_set = {t["produkt"] for t in transakcje if t["cena"] > 500}
print(f"\nUnikalne drogie produkty: {drogie_produkty_set}")

# List comprehension - transakcje o dużej wartości (>1000)
duze_transakcje = [
    {"id": t["id"], "produkt": t["produkt"], "wartosc": t["cena"] *
t["ilosc"]}
    for t in transakcje
    if t["cena"] * t["ilosc"] > 1000
]

print(f"\nDuże transakcje (wartość >1000 PLN):")
for trans in duze_transakcje:
    print(f"  ID {trans['id']}: {trans['produkt']} - {trans['wartosc']}
PLN")
```

Schemat typów comprehensions



Tip

Wybieraj odpowiedni typ comprehension w zależności od tego, czego potrzebujesz:

- **Lista**: zachowujesz kolejność i duplikaty
- **Słownik**: mapowanie klucz-wartość
- **Zbiór**: unikalne elementy, brak kolejności

🌱 Aspekt środowiskowy

W tej lekcji zwracamy uwagę na:

1. Funkcje wielokrotnego użytku = mniej duplikacji kodu

Definiując funkcje, unikamy powielania tego samego kodu w wielu miejscach projektu. To oznacza:

- **Mniej kodu do przetworzenia** przez interpreter
- **Szybsze uruchamianie** programów
- **Łatwiejsza optymalizacja** - zmiany w jednym miejscu
- **Mniejsze pliki źródłowe** - oszczędność pamięci i transferu danych

```
# ❌ Złe podejście - duplikacja kodu
# Obliczanie średniej w 5 różnych miejscach projektu - 50 linii kodu
srednia1 = sum(dane1) / len(dane1)
srednia2 = sum(dane2) / len(dane2)
# ... itd.

# ✅ Dobre podejście - funkcja wielokrotnego użytku
def srednia(dane):
    return sum(dane) / len(dane)

# Jedna funkcja, wiele wywołań - 10 linii kodu
# Mniej kodu = mniej energii do przetworzenia
```

2. List Comprehensions = wyższa wydajność

List comprehensions są zoptymalizowane na poziomie interpretera Pythona:

- **Szybsze wykonanie** niż pętle for (o 10-30%)
- **Mniejsze zużycie pamięci** - brak dodatkowych wywołań append()
- **Mniej instrukcji** - krótszy kod

```
import time

n = 1_000_000

# Pomiar: pętla for
start = time.time()
wynik1 = []
for i in range(n):
    wynik1.append(i * 2)
czas_petla = time.time() - start
```

```python
# Pomiar: comprehension
start = time.time()
wynik2 = [i * 2 for i in range(n)]
czas_comp = time.time() - start

print(f"Pętla for: {czas_petla:.3f}s")
print(f"Comprehension: {czas_comp:.3f}s")
print(f"Oszczędność czasu: {(1 - czas_comp/czas_petla) * 100:.1f}%")
# Typowo: Comprehension jest szybsze o 15-25%
```

3. Lambda funkcje = zwięzłość i czytelność

Krótszy kod to:

- **Mniej błędów** - mniejsza powierzchnia dla bugów
- **Szybsze przeglądanie** - mniej scrollowania
- **Łatwiejsze utrzymanie** - szybsze zrozumienie logiki

```python
# ❌ Długie podejście
def do_kwadratu(x):
    return x ** 2

kwadraty = list(map(do_kwadratu, range(10)))

# ✅ Zwięzłe podejście
kwadraty = list(map(lambda x: x ** 2, range(10)))
# Jeden wiersz zamiast czterech
```

4. Praktyczne zasady Green IT w funkcjach

```python
# Zasada 1: Funkcje z wartościami domyślnymi oszczędzają pamięć
def przetwarzaj_dane(dane, max_wierszy=1000):
    """
    Domyślnie przetwarzamy tylko 1000 wierszy - oszczędność pamięci.
    W fazie testowania nie potrzebujemy pełnych danych!
    """
    return dane[:max_wierszy]

# Zasada 2: Generatory zamiast list dla dużych danych
# ❌ Lista - cała w pamięci
duza_lista = [i ** 2 for i in range(10_000_000)]  # ~80 MB RAM

# ✅ Generator - elementy generowane na żądanie
duzy_generator = (i ** 2 for i in range(10_000_000))  # ~128 bajtów RAM
```

```python
# Zasada 3: Funkcje do czyszczenia zasobów
def zwolnij_pamiec(zmienne):
    """Usuwa duże zmienne z pamięci."""
    for zmienna in zmienne:
        del zmienna
    import gc
    gc.collect()  # wymuszamy garbage collection
```

Podsumowanie Green IT

Technika	Korzyść energetyczna	Oszczędność
Funkcje zamiast duplikacji	Mniej kodu do przetworzenia	10-50% mniej linii
Comprehensions vs pętle	Szybsze wykonanie	15-25% czasu
Lambda vs pełne funkcje	Mniejsze pliki	5-10% rozmiaru
Wartości domyślne	Próbkowanie danych	50-90% pamięci

> ✓ **Success**
>
> Pisząc idiomatyczny kod Python (funkcje, comprehensions, lambda), automatycznie piszesz kod bardziej efektywny energetycznie. Pythonowe idiomy to nie tylko elegancja - to też ekologia!

🤖 Narzędzia AI/ML wykorzystujące funkcje i idiomy

W przyszłych lekcjach będziemy często używać funkcji i comprehensions w kontekście bibliotek ML:

Scikit-learn - funkcje transformacji

```python
# Przykład: własne funkcje transformacji w pipeline'ach scikit-learn
from sklearn.preprocessing import FunctionTransformer

# Definiujemy funkcję transformacji
def log_transform(X):
    import numpy as np
    return np.log1p(X)  # log(1 + x)
```

```
# Używamy jej w pipeline ML
transformer = FunctionTransformer(log_transform)
# dane_transformed = transformer.fit_transform(dane)
```

Pandas - apply() z lambda

```
# Przykład: transformacje kolumn DataFrame z lambda
# import pandas as pd
# df['nowa_kolumna'] = df['stara_kolumna'].apply(lambda x: x * 2)
# df_filtered = df[df['kolumna'].apply(lambda x: x > 0)]
```

Feature engineering - comprehensions

```
# Przykład: tworzenie wielu cech jednocześnie
# cechy = ['wiek', 'dochod', 'wydatki']
# cechy_kwadrat = [f"{cecha}_squared" for cecha in cechy]
# for cecha, cecha_kw in zip(cechy, cechy_kwadrat):
#     df[cecha_kw] = df[cecha] ** 2
```

📊 Praca z Prawdziwymi Danymi

Dataset: Analiza wyników sprzedaży

Pracujemy z realistycznymi danymi sprzedażowymi - 30 dni sprzedaży w różnych kategoriach produktów.

```
# Dane sprzedażowe - dzień, kategoria, liczba transakcji, wartość
sprzedaz_raw = [
    (1, "elektronika", 45, 12500),
    (1, "odziez", 78, 4200),
    (1, "ksiazki", 34, 890),
    (2, "elektronika", 52, 15200),
    (2, "odziez", 65, 3800),
    (2, "ksiazki", 29, 760),
    (3, "elektronika", 38, 9800),
    (3, "odziez", 82, 5100),
    (3, "ksiazki", 41, 1020),
    (4, "elektronika", 61, 18900),
    (4, "odziez", 71, 4500),
    (4, "ksiazki", 38, 950),
    (5, "elektronika", 44, 11200),
```

```
    (5, "odziez", 88, 5600),
    (5, "ksiazki", 45, 1150)
]
# ... (dane kontynuowane dla 30 dni)

# Nazwy kolumn: (dzień, kategoria, liczba_transakcji, wartość_PLN)
```

Preprocessing - transformacja do słownika

```
# Funkcja konwertująca tuple na słownik
def tuple_to_dict(rekord):
    """Konwertuje tuple danych sprzedażowych na słownik."""
    return {
        "dzien": rekord[0],
        "kategoria": rekord[1],
        "liczba_transakcji": rekord[2],
        "wartosc_pln": rekord[3]
    }

# List comprehension - konwersja wszystkich rekordów
sprzedaz_dict = [tuple_to_dict(r) for r in sprzedaz_raw]

print("Przykładowe rekordy po konwersji:")
for rekord in sprzedaz_dict[:3]:
    print(f"  Dzień {rekord['dzien']}, {rekord['kategoria']}: "
          f"{rekord['liczba_transakcji']} transakcji, "
          f"{rekord['wartosc_pln']} PLN")
```

Feature Engineering - nowe cechy

```
# Dodajemy nowe cechy: średnia wartość transakcji
sprzedaz_engineered = [
    {
        **rekord,
        "srednia_wartosc_transakcji": round(
            rekord["wartosc_pln"] / rekord["liczba_transakcji"],
            2
        )
    }
    for rekord in sprzedaz_dict
]

print("\nRekordy z nową cechą 'średnia wartość transakcji':")
for rekord in sprzedaz_engineered[:3]:
```

```
    print(f"  {rekord['kategoria']}: średnia transakcja = "
          f"{rekord['srednia_wartosc_transakcji']} PLN")
```

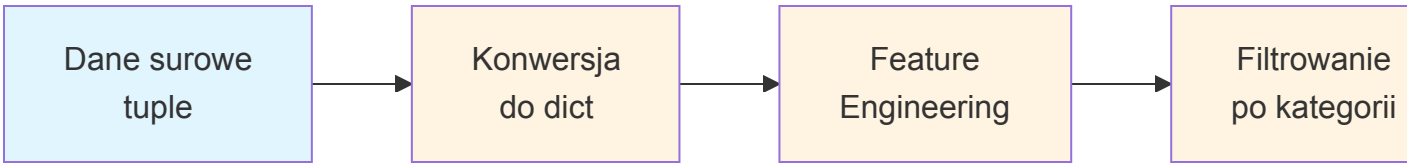
Analiza - agregacja po kategoriach

```
# Funkcja agregująca sprzedaż dla kategorii
def agreguj_kategorie(dane, kategoria):
    """
    Oblicza łączne statystyki dla danej kategorii.

    Returns:
        dict: statystyki (suma transakcji, suma wartości, średnia)
    """
    # Filtrujemy dane dla kategorii
    dane_kat = [r for r in dane if r["kategoria"] == kategoria]

    # Agregacje
    suma_transakcji = sum(r["liczba_transakcji"] for r in dane_kat)
    suma_wartosci = sum(r["wartosc_pln"] for r in dane_kat)
    srednia_dzienna = suma_wartosci / len(dane_kat)

    return {
        "kategoria": kategoria,
        "suma_transakcji": suma_transakcji,
        "suma_wartosci": suma_wartosci,
        "srednia_dzienna": round(srednia_dzienna, 2),
        "liczba_dni": len(dane_kat)
    }

# Unikalne kategorie - set comprehension
kategorie = {r["kategoria"] for r in sprzedaz_engineered}

# Agregacja dla każdej kategorii
statystyki_kategorii = [
    agreguj_kategorie(sprzedaz_engineered, kat)
    for kat in kategorie
]

print("\n=== Statystyki sprzedaży po kategoriach ===")
for stat in statystyki_kategorii:
    print(f"\n{stat['kategoria'].upper()}:")
    print(f"  Suma transakcji: {stat['suma_transakcji']}")
    print(f"  Suma wartości: {stat['suma_wartosci']} PLN")
    print(f"  Średnia dzienna: {stat['srednia_dzienna']} PLN")
```

Wizualizacja wyników

```
# Sortujemy kategorie według sumy wartości
statystyki_sorted = sorted(
    statystyki_kategorii,
    key=lambda s: s["suma_wartosci"],
    reverse=True
)

print("\n=== Ranking kategorii (według wartości sprzedaży) ===")
for i, stat in enumerate(statystyki_sorted, 1):
    print(f"{i}. {stat['kategoria']}: {stat['suma_wartosci']} PLN")
```

Pipeline przetwarzania danych



🧪 Zadania do samodzielnej pracy

✏️ Zadania podstawowe (1-8)

1. ✏️ **Zadanie 1 – Kalkulator statystyk**

   Napisz funkcję `statystyki(liczby)`, która przyjmuje listę liczb i zwraca tuple zawierający: (suma, średnia, minimum, maksimum).

   Dataset:

   - Lista ocen: `[3.5, 4.0, 5.0, 3.0, 4.5, 5.0, 4.0, 3.5]`

   Wymagania:

   - Funkcja powinna zwracać tuple z 4 wartościami
   - Średnia zaokrąglona do 2 miejsc po przecinku

   Oczekiwany rezultat:

   - Wypisanie wszystkich statystyk na ekran

   (proste)

2. ✏️ **Zadanie 2 – Konwerter jednostek**

   Napisz trzy funkcje: `celsius_to_fahrenheit(c)`, `fahrenheit_to_celsius(f)`, `celsius_to_kelvin(c)`.

   Dataset:

   - Temperatury w Celsius: `[0, 10, 20, 30, 37, 100]`

   Wymagania:

   - Każda funkcja zwraca skonwertowaną wartość
   - Zastosuj funkcje do konwersji wszystkich temperatur

   Oczekiwany rezultat:

   - Tabela z trzema kolumnami: °C, °F, K

   (proste)
3. ✏️ **Zadanie 3 – Filtr liczb parzystych**

   Używając list comprehension, stwórz listę zawierającą tylko liczby parzyste z zakresu 1-100.

   Dataset:

   - Zakres: 1-100

   Wymagania:

   - Użyj list comprehension z warunkiem
   - Wypisz pierwsze 10 liczb parzystych

   Oczekiwany rezultat:

   - Lista: `[2, 4, 6, 8, 10, ...]`

   (proste)
4. ✏️ **Zadanie 4 – Kwadraty liczb nieparzystych**

   Używając list comprehension, stwórz listę kwadratów liczb nieparzystych z zakresu 1-20.

   Dataset:

   - Zakres: 1-20

   Wymagania:

- List comprehension z warunkiem `x % 2 != 0`
- Oblicz kwadraty tych liczb

Oczekiwany rezultat:

- Lista: `[1, 9, 25, 49, ...]`

(proste)

5. ✏️ **Zadanie 5 – Lambda do sortowania**

   Masz listę tuple z produktami: `(nazwa, cena, ocena)`. Posortuj ją według ceny rosnąco, używając `sorted()` i lambda.

   Dataset:

```
produkty = [
    ("laptop", 2999, 4.5),
    ("mysz", 89, 4.2),
    ("klawiatura", 249, 4.8),
    ("monitor", 899, 4.3)
]
```

   Wymagania:

   - Użyj `sorted()` z parametrem `key=lambda ...`
   - Posortuj według drugiego elementu tuple (cena)

   Oczekiwany rezultat:

   - Lista posortowana od najtańszego do najdroższego

   (proste)

6. ✏️ **Zadanie 6 – Funkcja z wartościami domyślnymi**

   Napisz funkcję `powitanie(imie, jezyk="polski")`, która wyświetla powitanie w wybranym języku (polski, angielski, niemiecki).

   Dataset:

   - Brak (funkcja operuje na stringach)

   Wymagania:

   - Domyślny język to polski
   - Obsługa 3 języków

- Wywołaj funkcję 3 razy z różnymi parametrami

Oczekiwany rezultat:

- `powitanie("Anna")` → "Cześć, Anna!"
- `powitanie("Anna", "angielski")` → "Hello, Anna!"

(proste)

7. ✏️ **Zadanie 7 – Długości słów**

Masz listę słów. Używając list comprehension, stwórz nową listę zawierającą długości tych słów.

Dataset:

```
slowa = ["python", "data", "science", "machine", "learning", "AI"]
```

Wymagania:

- Użyj list comprehension: `[len(slowo) for slowo in slowa]`

Oczekiwany rezultat:

- Lista długości: `[6, 4, 7, 7, 8, 2]`

(proste)

8. ✏️ **Zadanie 8 – Map i filter**

Używając `map()` i `filter()` z lambda, przetwórz listę liczb: najpierw zostaw tylko dodatnie, potem podwój każdą.

Dataset:

```
liczby = [-5, 3, -2, 8, 0, -1, 7, 4, -9, 6]
```

Wymagania:

- `filter()` z lambda: `x > 0`
- `map()` z lambda: `x * 2`
- Konwertuj wynik na listę

Oczekiwany rezultat:

- Lista: `[6, 16, 14, 8, 12]` (tylko dodatnie i podwojone)

(proste)

✏️ Zadania średnie (9-12)

9. ✏️ **Zadanie 9 – Funkcja normalizacji min-max**

   Napisz funkcję `normalize_minmax(dane)`, która normalizuje listę liczb do zakresu [0, 1] używając metody min-max.

   Dataset:

   ```
   wynagrodzenia = [3500, 4200, 5800, 7200, 9500, 4800, 6100]
   ```

   Wymagania:

   - Wzór: `(x - min) / (max - min)`
   - Funkcja zwraca nową listę znormalizowanych wartości
   - Zaokrąglij do 3 miejsc po przecinku

   Oczekiwany rezultat:

   - Lista wartości w zakresie [0.0, 1.0]

   (średnie)

10. ✏️ **Zadanie 10 – Dictionary comprehension - agregacja**

    Masz listę transakcji: `[{"produkt": "X", "wartosc": 100}, ...]`. Używając dict comprehension i grupowania, oblicz łączną wartość dla każdego produktu.

    Dataset:

    ```
    transakcje = [
        {"produkt": "laptop", "wartosc": 2999},
        {"produkt": "mysz", "wartosc": 89},
        {"produkt": "laptop", "wartosc": 2999},
        {"produkt": "klawiatura", "wartosc": 249},
        {"produkt": "mysz", "wartosc": 89},
        {"produkt": "mysz", "wartosc": 89}
    ]
    ```

    Wymagania:

    - Pogrupuj po produkcie
    - Zsumuj wartości dla każdego produktu

- Wynik jako słownik: `{"laptop": 5998, ...}`

Oczekiwany rezultat:

- Słownik z sumą wartości per produkt

(średnie)

11. ✏️ **Zadanie 11 – Feature engineering z comprehension**

Masz dane klientów: wiek i dochód. Dodaj nowe cechy: `kat_wiekowa` (młody/średni/starszy) i `kat_dochodu` (niski/średni/wysoki).

Dataset:

```
klienci = [
    {"id": 1, "wiek": 25, "dochod": 4500},
    {"id": 2, "wiek": 45, "dochod": 9200},
    {"id": 3, "wiek": 32, "dochod": 6800},
    {"id": 4, "wiek": 28, "dochod": 5200},
    {"id": 5, "wiek": 55, "dochod": 12000}
]
```

Wymagania:

- Kategoria wiekowa: <30 = młody, 30-50 = średni, >50 = starszy
- Kategoria dochodu: <5000 = niski, 5000-10000 = średni, >10000 = wysoki
- Użyj list comprehension do tworzenia nowej listy z dodatkowymi cechami

Oczekiwany rezultat:

- Lista klientów z 2 nowymi cechami

(średnie)

12. ✏️ **Zadanie 12 – Zagnieżdżone comprehensions - macierz**

Stwórz macierz 10x10 zawierającą wartości `i * j` (iloczyn indeksów) używając zagnieżdżonych list comprehensions.

Dataset:

- Rozmiar macierzy: 10x10

Wymagania:

- Zewnętrzna comprehension: wiersze (i od 1 do 10)

- Wewnętrzna comprehension: kolumny (j od 1 do 10)
- Wartość: `i * j`

Oczekiwany rezultat:

- Macierz 10x10 wydrukowana czytelnie

(średnie)

🧠 Zadania wyzwanie (13-20)

13. 🧠 **Zadanie 13 – Pipeline przetwarzania danych sprzedażowych**

Stwórz kompletny pipeline do analizy danych sprzedażowych. Dane zawierają: dzień, kategoria, liczba transakcji, wartość.

Dataset:

```
sprzedaz = [
    (1, "elektronika", 45, 12500),
    (1, "odziez", 78, 4200),
    (2, "elektronika", 52, 15200),
    # ... więcej danych dla 7 dni
]
```

Wymagania:

- Funkcja `tuple_to_dict()` - konwersja tuple na dict
- Funkcja `dodaj_srednia_transakcje()` - feature engineering
- Funkcja `agreguj_po_kategorii()` - suma wartości per kategoria
- Użyj comprehensions do przetwarzania
- Sortowanie kategorii według wartości (malejąco)

Oczekiwany rezultat:

- Ranking kategorii z sumą wartości sprzedaży

(challenge)

14. 🧠 **Zadanie 14 – Funkcja do detekcji outliers**

Napisz funkcję `wykryj_outliery(dane, metoda="IQR")`, która wykrywa wartości odstające używając metody IQR (Interquartile Range).

Dataset:

```
czasy_odpowiedzi = [120, 135, 142, 128, 5000, 138, 145, 119, 10000, 133,
141]
```

Wymagania:

- Oblicz Q1 (25 percentyl) i Q3 (75 percentyl)
- IQR = Q3 - Q1
- Outliers: wartości < Q1 - 1.5*IQR lub > Q3 + 1.5*IQR
- Funkcja zwraca (czyste_dane, outliery)

Oczekiwany rezultat:

- Lista outlierów: `[5000, 10000]`
- Lista czystych danych bez outlierów

(challenge)

15. 🧠 **Zadanie 15 – Funkcje wyższego rzędu - decorator pattern**

Napisz funkcję `timer(func)`, która mierzy czas wykonania innej funkcji. Użyj jej do porównania wydajności pętli vs comprehension.

Dataset:

- Zakres: 1-100000

Wymagania:

- Funkcja `timer()` przyjmuje funkcję jako argument
- Wewnątrz `timer()` mierz czas wykonania `func()`
- Porównaj czas dla:
  - Funkcji z pętlą for tworzącą listę kwadratów
  - List comprehension tworzącej listę kwadratów

Oczekiwany rezultat:

- Wydruk: "Pętla: 0.123s, Comprehension: 0.089s"

(challenge)

16. 🧠 **Zadanie 16 – Rekurencja - obliczanie silni i Fibonacciego**

Napisz dwie funkcje rekurencyjne: `silnia(n)` i `fibonacci(n)`.

Dataset:

- n = 10 dla silni
- n = 15 dla Fibonacciego

Wymagania:

- Silnia: `n! = n * (n-1)!`, przypadek bazowy: `0! = 1`
- Fibonacci: `fib(n) = fib(n-1) + fib(n-2)`, bazowe: `fib(0)=0, fib(1)=1`
- Oblicz silnię dla 10
- Oblicz pierwsze 15 liczb Fibonacciego

Oczekiwany rezultat:

- Silnia(10) = 3628800
- Fibonacci(15): [0, 1, 1, 2, 3, 5, 8, 13, 21, ...]

(challenge)

17. 🧠 **Zadanie 17 – Generator vs Lista - oszczędność pamięci**

Porównaj zużycie pamięci: lista vs generator dla dużych danych.

Dataset:

- n = 10 milionów elementów

Wymagania:

- Stwórz list comprehension: `[i**2 for i in range(n)]`
- Stwórz generator: `(i**2 for i in range(n))`
- Użyj `sys.getsizeof()` do pomiaru rozmiaru w bajtach
- Oblicz różnicę w MB

Oczekiwany rezultat:

- Lista: ~80 MB
- Generator: ~128 bajtów
- Oszczędność pamięci: >99%

(challenge)

18. 🧠 **Zadanie 18 – Funkcja kroswalidacji (podstawowa wersja)**

Napisz funkcję `k_fold_split(dane, k)`, która dzieli dane na k części (foldów) do walidacji krzyżowej.

Dataset:

```
dane = list(range(1, 101))  # 100 obserwacji
```

Wymagania:

- k = 5 (5-fold cross-validation)
- Podziel dane na 5 równych części
- Funkcja zwraca listę foldów (każdy fold to lista)
- Użyj slicing i comprehensions

Oczekiwany rezultat:

- 5 foldów po 20 elementów każdy
- Fold 1: [1-20], Fold 2: [21-40], ...

(challenge)

19. 🧠 **Zadanie 19 – Pipeline z funkcjami: czyszczenie → transformacja → agregacja**

Stwórz kompletny pipeline analizy danych klientów: czyszczenie outlierów → normalizacja → segmentacja.

Dataset:

```
klienci = [
    {"id": 1, "wiek": 25, "dochod": 4500, "wydatki": 3200},
    {"id": 2, "wiek": 200, "dochod": 7800, "wydatki": 5100},  # outlier
wiek
    {"id": 3, "wiek": 45, "dochod": 9200, "wydatki": -500},   # outlier
wydatki
    # ... więcej danych
]
```

Wymagania:

- Funkcja `usun_outliery()` - usuwa rekordy z błędnymi wartościami
- Funkcja `normalizuj_kolumny()` - normalizacja min-max dla wieku/dochodu
- Funkcja `segmentuj()` - przypisuje klientów do segmentów (niski/średni/wysoki) według dochodu
- Użyj comprehensions i lambda w pipeline

Oczekiwany rezultat:

- Czyste dane z segmentami i znormalizowanymi wartościami

(challenge)

20. 🧠 **Zadanie 20 – Własna implementacja map() i filter()**

    Napisz własne wersje funkcji `my_map(func, iterable)` i `my_filter(predicate, iterable)` bez używania wbudowanych.

    Dataset:

    ```
    liczby = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    ```

    Wymagania:

    - `my_map()` aplikuje funkcję do każdego elementu
    - `my_filter()` zwraca tylko elementy spełniające warunek
    - Użyj pętli for i append
    - Przetestuj: podwój liczby, następnie zostaw tylko parzyste

    Oczekiwany rezultat:

    - `my_map(lambda x: x*2, liczby)` → [2, 4, 6, ..., 20]
    - `my_filter(lambda x: x%2==0, ...)` → [2, 4, 6, 8, 10]

    (challenge)

📚 Podsumowanie

W tej lekcji poznaliśmy kluczowe techniki programowania w Pythonie, które są fundamentem pracy z danymi:

Czego się nauczyliśmy:

1. **Funkcje**
   - Definiowanie funkcji z parametrami i wartościami domyślnymi
   - Zwracanie pojedynczych wartości i tuple
   - Docstringi do dokumentacji
   - Modularyzacja kodu - funkcje wielokrotnego użytku
2. **Funkcje lambda**
   - Składnia `lambda argumenty: wyrażenie`
   - Użycie z `map()`, `filter()`, `sorted()`
   - Transformacje danych w jednej linii

- Feature engineering z lambda

3. **List Comprehensions**
   - Składnia `[wyrażenie for element in sekwencja if warunek]`
   - Filtrowanie i transformacja jednocześnie
   - Zagnieżdżone comprehensions dla struktur 2D
   - Wyższa wydajność niż tradycyjne pętle
4. **Dictionary i Set Comprehensions**
   - `{klucz: wartość for ... in ...}` dla słowników
   - `{wyrażenie for ... in ...}` dla zbiorów
   - Mapowanie i agregacja danych
   - Usuwanie duplikatów

Jak to się łączy z poprzednimi lekcjami:

- **Lekcje 4-7 (Python basics)**: Funkcje i idiomy to rozszerzenie poznanych już konstrukcji (zmienne, pętle, kolekcje)
- **Kontrola przepływu**: Funkcje enkapsulują logikę z if/elif/else i pętli
- **Kolekcje**: Comprehensions działają na listach, słownikach i zbiorach poznanych wcześniej

Przygotowanie do następnych tematów:

Poznane techniki będą intensywnie używane w kolejnych lekcjach:

- **NumPy i Pandas**: funkcje transformacji DataFrame'ów, apply(), map()
- **Feature engineering**: tworzenie nowych cech z istniejących
- **Machine Learning**: własne funkcje preprocessing, custom transformers
- **Pipeline ML**: łączenie kroków obróbki danych w funkcjonalne pipeline'y

Refleksja nad aspektem środowiskowym:

Pisząc idiomatyczny kod Python (funkcje, comprehensions, lambda):

- **Oszczędzamy energię** - mniej kodu, szybsze wykonanie
- **Redukujemy duplikację** - funkcje wielokrotnego użytku
- **Optymalizujemy pamięć** - generatory zamiast list tam, gdzie to możliwe
- **Ułatwiamy utrzymanie** - mniej bugów = mniej debugowania = mniej zużytych zasobów

✓ **Success**

Pythonowe idiomy to nie tylko elegancki kod - to również ekologiczny kod. Pisząc w stylu "Pythonic", automatycznie dbasz o efektywność energetyczną swoich programów!

📖 Dodatkowe Zasoby

Dokumentacja

- Python Functions - dokumentacja oficjalna
- Lambda Expressions
- List Comprehensions

Artykuły i tutoriale

- Real Python: "Python Lambda Functions" - praktyczne przykłady
- Real Python: "When to Use a List Comprehension in Python" - best practices
- PEP 8 - Style Guide for Python Code - konwencje nazewnictwa funkcji

Papers i badania

- "A Performance Comparison of Python List Comprehensions vs. For Loops" - analiza wydajności
- "Green Software Engineering" - wpływ idiomatycznego kodu na zużycie energii

Polecane datasety do praktyki

- Kaggle Datasets - ogromna kolekcja danych do ćwiczeń
- UCI Machine Learning Repository - klasyczne datasety (Iris, Wine, Boston Housing)
- data.gov - otwarte dane rządowe (USA)
- dane.gov.pl - otwarte dane rządowe (Polska)

Narzędzia

- PythonTutor - wizualizacja wykonania kodu krok po kroku
- timeit - precyzyjny pomiar czasu wykonania
- memory_profiler - analiza zużycia pamięci

Gratulacje! Ukończyłeś Lekcję 08 i opanowałeś kluczowe techniki idiomatycznego Pythona. Jesteś gotowy do pracy z zaawansowanymi bibliotekami do analizy danych! 🚀